
Fundamentals – Distributed Algorithms

Replication and Consistency

Outline

- Introduction
- Data-Centric Consistency Models and Protocols
- Client-Centric Consistency Models and Protocols

Introduction

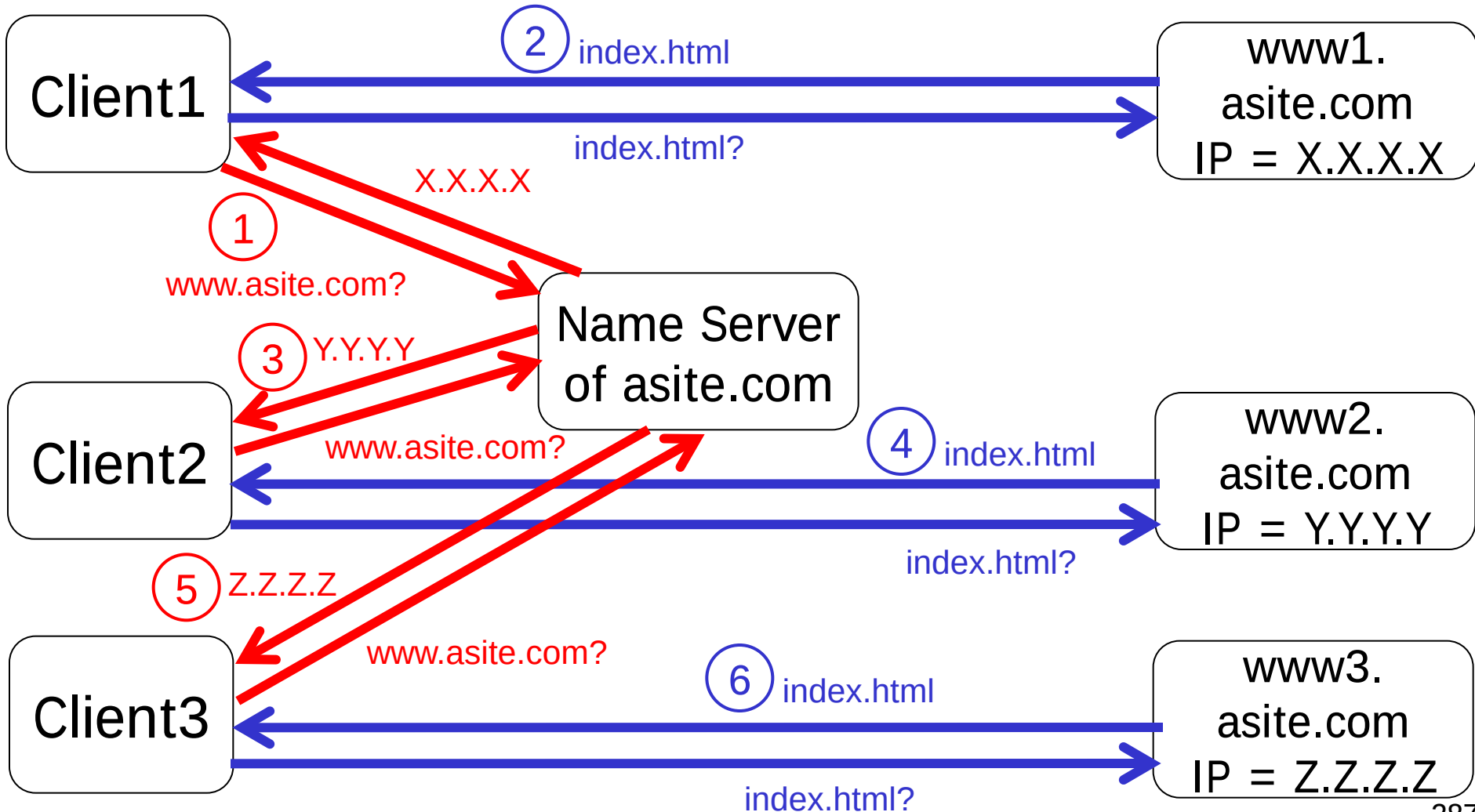
- Data replication: maintaining copies of data at multiple computers
- Motivations for replication
 - Improve performance and scalability
 - Increase reliability and fault-tolerance
- Price to pay: keeping all replicas consistent

Benefits of Replication

- Improve performance and scalability
 - Replication for performance is important when a distributed system needs to scale in numbers and geographical area
 - Example: several web servers can have the same domain name and the servers are selected in turn for the purpose of sharing load
 - By placing a copy of data in the proximity of the process using them, the time to access the data decreases

Benefits of Replication

- Improve performance and scalability (cont'd)



Benefits of Replication

- Increase reliability and fault tolerance
 - A service's availability is hindered by server failures
 - Replicate data at failure-independent servers
 - When one server crashes, clients may use another server
 - Example: primary and secondary servers in DNS
 - How to measure availability? – the portion of time the service/system is available for use
 - If each of n servers has an independent probability p of crashing, then the availability of an object stored in all of these servers is

$$1 - \text{Prob}(\text{all servers crash}) = 1 - p^n$$

$$\text{e.g., } p = 10\%, \quad n = 1, \text{ availability} = 90\%$$

$$n = 2, \text{ availability} = 99\% \quad n = 3, \text{ availability} = 99.9\%$$

Benefits of Replication

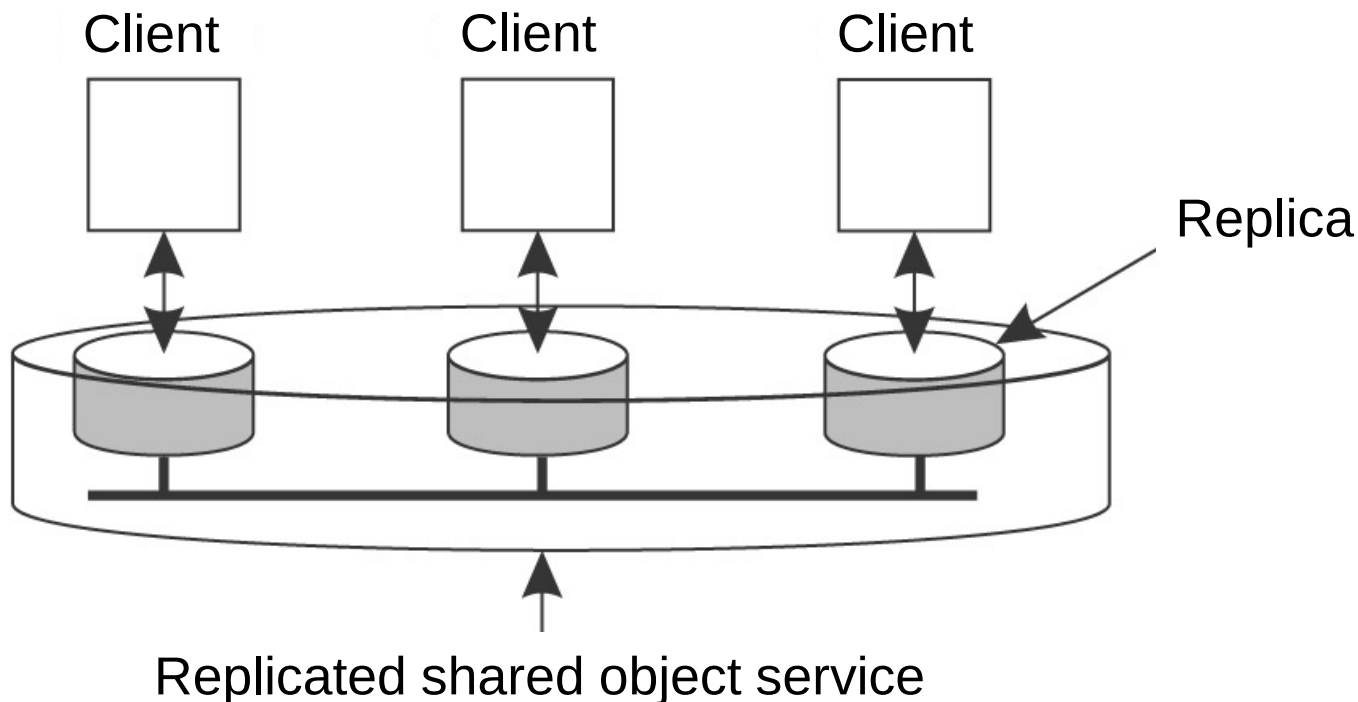
- Increase reliability and fault tolerance (cont'd)
 - Availability is also hindered by network partitions and disconnected operations
 - Users often copy heavily used data, such as the contents of a shared diary, from their usual environment to the laptops
 - They can then access the data on the laptops even if they are disconnected from the network
 - Problem – What if several users update the local copies of the shared diary on their laptops respectively? How to prevent a user from missing the updates by other people?

Problem with Replication

- Having multiple copies of data leads to consistency problems
 - Whenever a copy is modified, that copy becomes different from the rest
 - → Modifications have to be carried out on all copies to ensure consistency
 - When and how those modifications need to be carried out determines the price of replication
 - Example: web browsers cache web pages
 - To let the web server invalidate cached copies when a web page is updated, it requires the server to keep track of all browser caches and send them messages
→ this may degrade the performance of the server

System Model

- Each **logical data object** is implemented by a collection of **physical copies** called **replicas**



System Model

- Static vs. Dynamic

- A **static system** contains a fixed set of replicas
- In a **dynamic system**, replicas may be created or destroyed on the fly (not considered in our lecture)

- Transparency

- Clients are not aware of multiple physical copies
- Clients see logical objects only (i.e., a service that gives them access to logical objects) – access one logical object and receive a single result for each operation

System Model

- Data operations

- **Read operation** – an operation that does not change the data
- **Write operation** (or update operation) – an operation that changes the data
- Operations are applied to the replicas **atomically**

- Possible failures

- Crash (process omission failure)
- Byzantine failure of processes
- Network partition

System Model

- Consistency model

- Normally, a read operation is expected to return a value that shows the result of the last write operation
- But due to inevitable network delay, replicas are not necessarily consistent all the time
 - Some replicas may have received updates that are not yet conveyed to the others → different replicas are likely to have different values
- Consistency models
 - Define what is “correct” behaviour
 - Concern whether operations performed on a collection of replicated objects produce results that meet the correctness criteria for these logical objects

Outline

- Introduction
- Data-Centric Consistency Models and Protocols
- Client-Centric Consistency Models and Protocols
- Summary

Data-Centric Consistency Models

- Target scenario
 - Concurrent processes may simultaneously update data objects
- Objective
 - To provide a system wide consistent view of data objects in the face of such concurrency

Strict Consistency

- Strict Consistency
 - Any read operation on a data object returns a value corresponding to the result of the **most recent** write operation on the object
- All writes are instantaneously visible to all clients
- Examples in this section
 - A replicated shared object service hosting two objects x and y whose initial values are 0
 - Data operations are performed by multiple clients
 - **read(x)** $\rightarrow a$: a read operation on x returning a value a
 - **write(x, b)** : a write operation setting the value of x to b

Strict Consistency

- Example of a service that is not strictly consistent

Client A	Client B
write($x,1$)	
write($y,2$)	
	read(y) $\rightarrow$ 2
	read(x) $\rightarrow$ 0

$\neq$

Interleaving
write($x,1$)
write($y,2$)
read(y) $\rightarrow$ 2
read(x) $\rightarrow$ 1

- Example of a strictly consistent service

Client A	Client B
	read(x) $\rightarrow$ 0
write($x,1$)	
write($y,2$)	
	read(y) $\rightarrow$ 2

$=$

Interleaving
read(x) $\rightarrow$ 0
write($x,1$)
write($y,2$)
read(y) $\rightarrow$ 2

Strict Consistency

■ Practicality

- Strict consistency assumes the existence of absolute global time
- In the absence of a global clock in distributed systems
 - It is difficult to assign a unique timestamp to each operation that corresponds to actual global time
 - It is difficult to define precisely which write operation is the “most recent”
- A client updates an object at a replica at time T_1 ; slightly later, another client reads the object from a different replica at time T_2
 - If $T_2 - T_1$ is extremely small, is it possible for the second client to read the new value of the object?

Sequential Consistency

- Strict consistency may not be practical
- A weaker model is **sequential consistency**
- Sequential consistency
 - The result of any execution is the same as if the (read and write) operations by all clients were executed in some sequential order and the operations of each individual client appear in this sequence **in the order specified by its program**

Interleaving

- Simply speaking, **an interleaving is a serialized sequence of the operations by all clients**
- Consider a replicated service with two clients
 - Client A performs operations o_{A0}, o_{A1}, o_{A2}
 - Client B performs operations o_{B0}, o_{B1}, o_{B2}
 - Example interleavings:
 - $o_{B0}, o_{B1}, o_{A0}, o_{B2}, o_{A1}, o_{A2}$
 - $o_{B2}, o_{A1}, o_{A2}, o_{B0}, o_{B1}, o_{A0}$

Interleaving

- Interleaving in program order
 - The order of operations issued by each client is preserved in the interleaving
 - Eligible interleaving may not be unique
 - Client A performs operations o_{A0}, o_{A1}, o_{A2}
 - Client B performs operations o_{B0}, o_{B1}, o_{B2}
 - Interleavings in program order:
 - $o_{B0}, o_{B1}, o_{A0}, o_{B2}, o_{A1}, o_{A2}$
 - $o_{A0}, o_{B0}, o_{B1}, o_{A1}, o_{A2}, o_{B2}$
 - Interleavings not in program order:
 - $o_{B2}, o_{A1}, o_{A2}, o_{B0}, o_{B1}, o_{A0}$
 - $o_{A1}, o_{A2}, o_{B0}, o_{B1}, o_{A0}, o_{B2}$

Sequential Consistency

- How to check?
- A replicated shared object service is **sequentially consistent** if there exists some interleaving of clients' operations such that
 - The order of operations in the interleaving is **consistent with the program order** in which each client executed them
 - The results returned by the replicated service match those of the interleaving on a single copy of objects

Sequential Consistency

- Example of a sequentially consistent service

Client A	Client B		Interleaving
write($x,1$)			read(y) $\rightarrow$ 0
	read(y) $\rightarrow$ 0		read(x) $\rightarrow$ 0
	read(x) $\rightarrow$ 0		write($x,1$)
write($y,2$)			write($y,2$)

Sequential Consistency

- Example of a not sequentially consistent service

Client A	Client B
write($x,1$)	
write($y,2$)	
	read(y) $\rightarrow$ 2
	read(x) $\rightarrow$ 0

For any interleaving in program order, write($x,1$) precedes write($y,2$), and read(y) precedes read(x).

If there exists an interleaving showing sequential consistency, since read(y) returns 2, write($y,2$) must precede read(y) in the interleaving. Thus, write($x,1$) precedes read(x). However, this is impossible since read(x) returns 0 in the replicated service.

Sequential Consistency

- Any interleaving of read and write operations in program order is acceptable, but all processes see the same interleaving of operations
- Any strictly consistent service is sequentially consistent
 - How to prove it?

Causal Consistency

- **Causal consistency** is an even weaker model that makes a distinction between events that are potentially causally related and those that are not
- Causal relation
 - Operations by a single client are causally related based on the order in which the client executed them
 - A read is causally related to the write that provided the data the read got
 - Not causally related → concurrent

Causal Consistency

- Example of causal relation

Client A	Client B	Client C	Client D
write(x,1)			
	write(x,2)		
		read(x) → 2	
		read(x) → 1	
			read(x) → 1
			read(x) → 2

The diagram illustrates causal relations between operations of four clients (A, B, C, D) using a table. Red arrows indicate dependencies between operations across different clients.

- Client A: write(x,1)
- Client B: write(x,2)
- Client C: read(x) → 2, read(x) → 1
- Client D: read(x) → 1, read(x) → 2

Red arrows show the following dependencies:

- Client A's write(x,1) is read by Client C's read(x) → 1 and Client D's read(x) → 1.
- Client B's write(x,2) is read by Client C's read(x) → 2 and Client D's read(x) → 2.
- Client C's read(x) → 1 is also read by Client D's read(x) → 1.

Causal Consistency

- Causal consistency
 - Writes that are potentially causally related must be seen by all clients in the same order
 - Concurrent writes may be seen in different orders by different clients

Causal Consistency

- How to check?
- A replicated shared object service is **causally consistent** if for each client, there exists some interleaving of this client's operations and other clients' write operations such that
 - The order of this client's operations in the interleaving is consistent with its program order
 - The order of operations in the interleaving is **consistent with the causal relations between operations**
 - The results returned by the replicated service match those of the interleaving on a single copy of objects

Causal Consistency

- Example of a causally consistent service

Client A	Client B	Client C	Client D
write($x,1$)			Client D sees: write($x,1$) read(x) $\rightarrow$ 1 write($x,2$) read(x) $\rightarrow$ 2
	write($x,2$)		
	Client C sees: write($x,2$) read(x) $\rightarrow$ 2 write($x,1$) read(x) $\rightarrow$ 1	read(x) $\rightarrow$ 2	
		read(x) $\rightarrow$ 1	
			read(x) $\rightarrow$ 1
			read(x) $\rightarrow$ 2

write($x,1$) and write($x,2$) are concurrent $\rightarrow$ it is not required that all clients see them in the same order.

However, the service is not sequentially consistent. Why?

Causal Consistency

- Example of a not causally consistent service

Client A	Client B	Client C	Client D
write(x,1)			
	read(x) → 1		
	write(x,2)		
		read(x) → 2	
		read(x) → 1	
			read(x) → 1
			read(x) → 2

write(x,1) and write(x,2) are potentially causally related
(write(x,2) may depend on read(x) which in turn depends on
write(x,1)) → a required interleaving for client C cannot be found

FIFO Consistency

- FIFO consistency
 - Writes done by a single client are seen by all other clients in the order in which they were issued
 - Writes from different clients may be seen in different orders by different clients

FIFO Consistency

- How to check?
- A replicated shared object service is **FIFO consistent** if for each client, there exists some interleaving of this client's operations and other clients' write operations such that
 - The order of this client's operations in the interleaving is consistent with its program order
 - The order of write operations in the interleaving is **consistent with the issuing order by each client**
 - The results returned by the replicated service match those of the interleaving on a single copy of objects

FIFO Consistency

- Example of an FIFO consistent service

Client A	Client B	Client C	Client D
write(x,1)			Client D sees: write(x,1) read(x) → 1 write(x,2) read(x) → 2
	read(x) → 1		
	write(x,2)		
		read(x) → 2	
	Client C sees: write(x,2) read(x) → 2 write(x,1) read(x) → 1	read(x) → 1	
			read(x) → 1
			read(x) → 2

write(x,1) and write(x,2) are from different clients → they may be seen in different orders by different clients.

FIFO Consistency

- Example of a not FIFO consistent service

Client A	Client B	Client C
write(x ,1)		
write(x ,2)		
	read(x) $\rightarrow$ 2	
	read(x) $\rightarrow$ 1	
		read(x) $\rightarrow$ 1
		read(x) $\rightarrow$ 2

write(x ,1) and write(x ,2) are from the same client $\rightarrow$ they are required to be seen by all other clients in their issuing order $\rightarrow$ a required interleaving for client B cannot be found.

Comparison

- **Strict consistency**
 - Most restrictive and stringent – absolute time ordering of all operations
- **Sequential consistency**
 - All clients see all operations in the same order, but the operations may not be ordered in absolute time
- **Causal consistency and FIFO consistency**
 - Different clients may see operations in different orders
 - Causal consistency – all clients see causally-related operations in the same order
 - FIFO consistency – all clients see writes from each other in the order they were issued

Consistency Protocols

- A consistency protocol describes an actual implementation of a specific consistency model
- We will also look at fault tolerant aspects
 - Does it handle faults?
 - What type of faults does it handle?
 - How many faults can it tolerate?

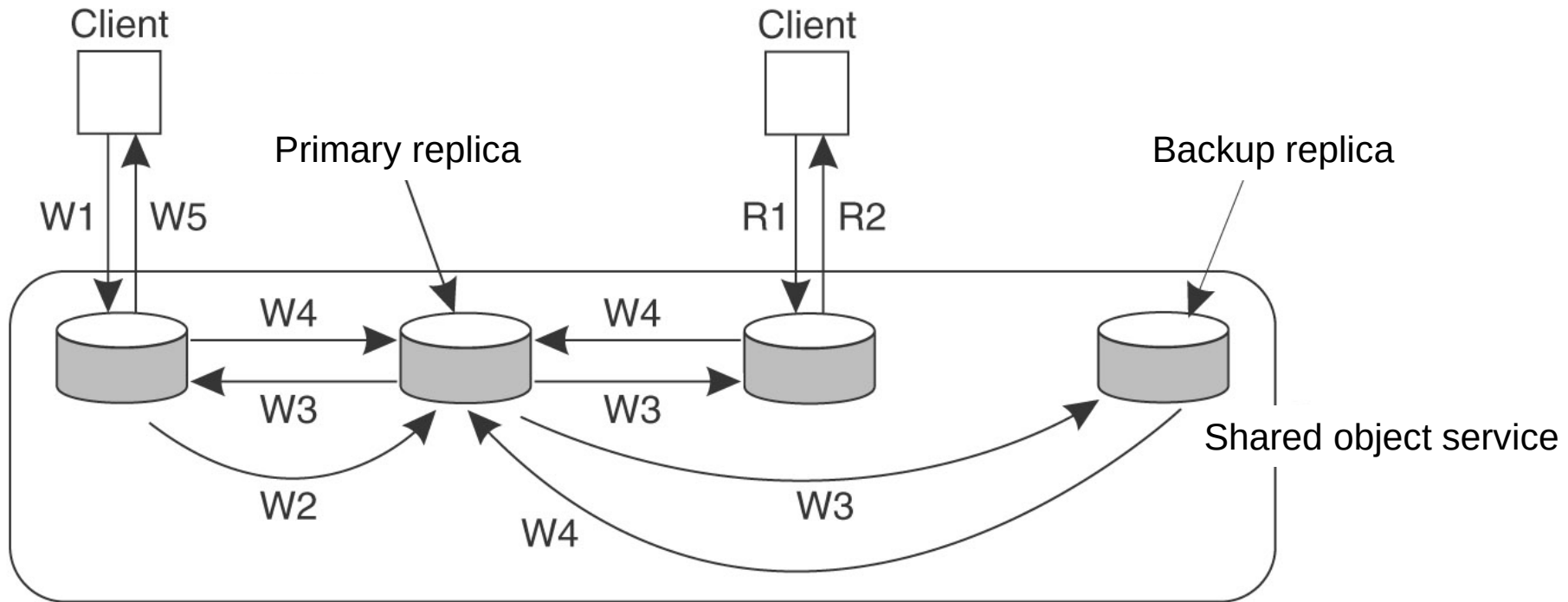
Primary-Based Protocols

- For each data object, one replica is designated as the **primary replica** and the remaining replicas are called **backups** (slaves or secondary replicas)
 - Known as passive replication
 - The primary replica is responsible for coordinating write operations on the object
 - If the primary replica crashes, one of the backups is elected to act as the primary replica
- Primary-based protocols
 - Remote-write protocol: the primary replica is fixed
 - Local-write protocol: the primary replica can be moved for write operations to be carried out locally

Remote-Write Protocol

- Write operations are forwarded to the primary replica
 - The primary replica performs the update and then forwards the update to the backup replicas
 - Each backup replica performs the update as well and sends an acknowledgement back to the primary replica
 - Finally, the primary replica sends an acknowledgment back to the initiating client
 - Write operations are blocking: a client initiating a write operation is not allowed to continue until it receives an acknowledgement from the primary replica
- Read operations can be carried out at any replica

Remote-Write Protocol



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

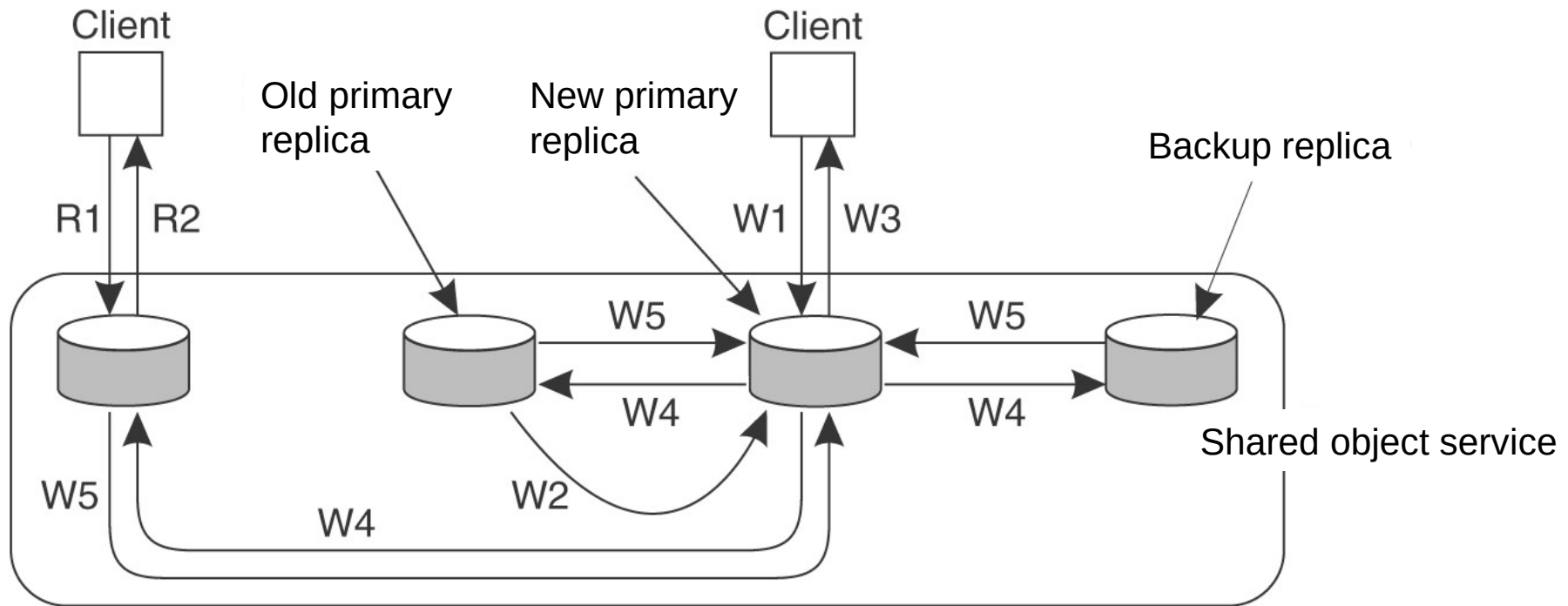
Remote-Write Protocol

- A straightforward implementation of sequential consistency
 - The primary replica orders all write operations in a globally unique order
 - So, all clients see all write operations in the same order, no matter which backup replica they use to perform read operations
 - Also, due to blocking, clients will always see the effects of their most recent write operations

Local-Write Protocol

- The primary migrates between replicas that wish to perform a write operation
- Write operations can be non-blocking
 - As soon as the primary replica has updated its local copy, it returns an acknowledgement to the client
 - After that, the primary replica tells the backup replicas to perform the update as well
- Advantages:
 - Multiple successive write operations can be carried out locally
 - Enable mobile clients to operate in disconnected mode

Local-Write Protocol



W1. Write request
W2. Move primary replica
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

Passive Replication

- Fault tolerance

- It can handle crashes
- To tolerate up to f process crashes, $f + 1$ replicas are required
- It cannot deal with byzantine failures because only one replica executes the operations

Replicated-Write Protocols

- Write operations can be carried out at multiple replicas instead of only one
- Active replication
- Gifford's quorum-based protocol

Active Replication

- All replicas play **the same role**
- Write operations are carried out at all replicas
 - Operations are sent to all replicas via **totally ordered multicast** (which delivers operations to all replicas in the same total order)
 - All replicas start in the same state and **perform the same operations in the same order** → their states remain identical
 - Write operations are blocking: a client initiating a write operation is not allowed to continue until it receives acknowledgements from all replicas
 - If a replica crashes, no action is needed by other replicas
- Read operations can be carried out at any replica

Why is Ordering Important?

Balance = \$1000

Receive 'deposit \$100'

Balance = \$1100

Receive 'add 1% interest'

Balance = \$1111

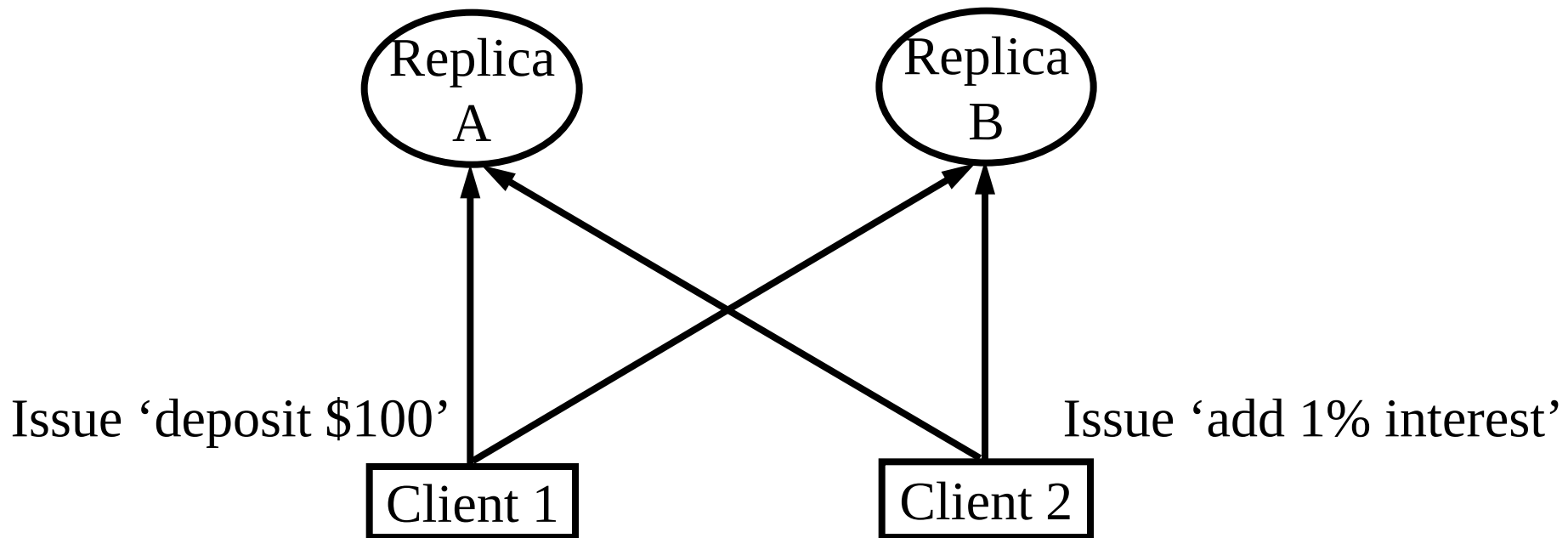
Balance = \$1000

Receive 'add 1% interest'

Balance = \$1010

Receive 'deposit \$100'

Balance = \$1110



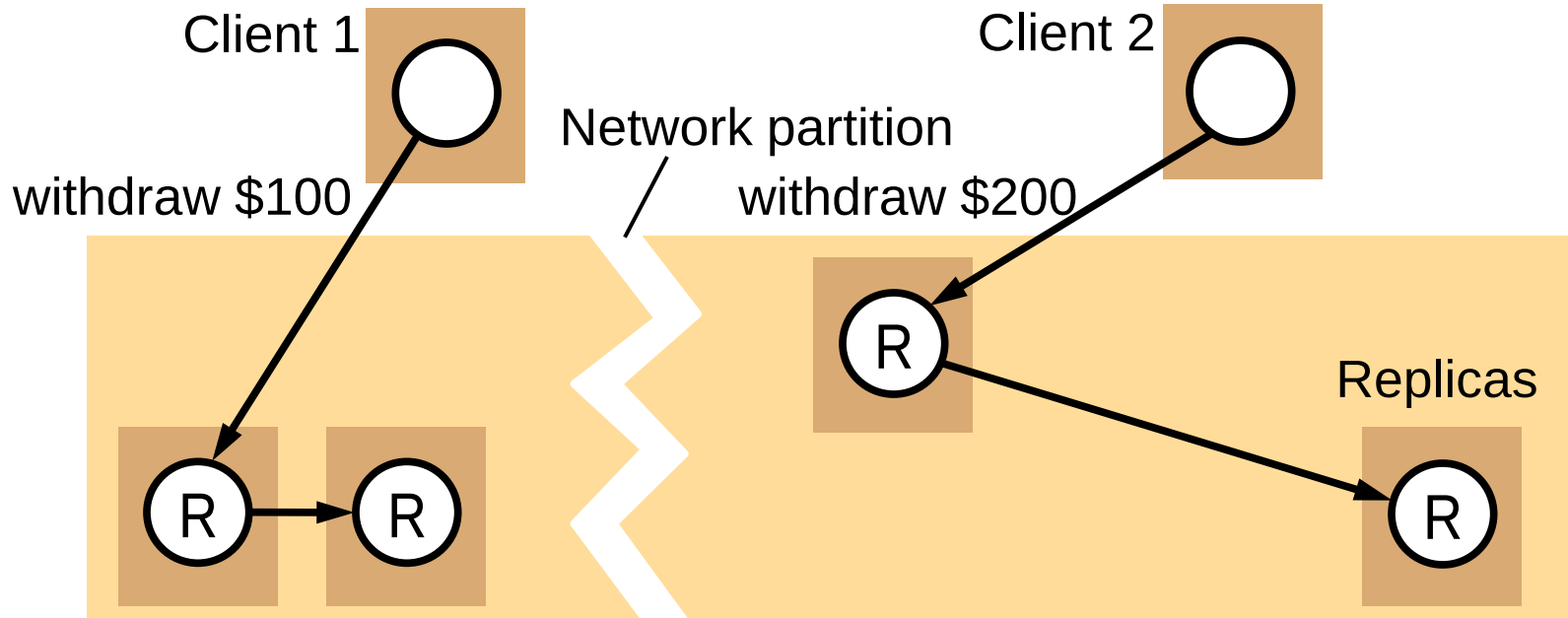
Active Replication

- Active replication achieves sequential consistency
 - Totally ordered multicast orders all write operations
- Fault tolerance
 - To tolerate crashes, the client only needs to wait for the first response
 - It can also handle byzantine failures
 - To tolerate up to f byzantine failures, $2f + 1$ replicas are required
 - The client waits for $f + 1$ identical responses

Network Partitions

- So far, we have assumed no network partition, what if there are network partitions?
- A **network partition** separates a group of replicas into two or more subgroups in such a way that
 - Replicas of one subgroup can communicate with one another
 - But replicas of different subgroups cannot communicate with one another
- Assumption: partitions will eventually be repaired
 - We must ensure that any operation executed during partition **will not make the replicas inconsistent** when the partition is repaired

Example of Network Partitions



- The replicas doing the left withdraw cannot communicate with those doing the right withdraw
- In general, reading during a partition would not cause inconsistency, but writing might

Handling Network Partitions

- Optimistic approach

- Allow updates in **all partitions** → this can lead to inconsistencies between partitions
- **Resolve inconsistencies** when the partition is repaired
- Unsuitable for some applications like banking

- Pessimistic approach

- Allow updates in **one partition only** → to prevent inconsistencies from occurring
- **Update the remaining replicas** when the partition is repaired
- Now let's look at Gifford's quorum-based protocol

Gifford's Quorum-Based Protocol

- Replicas in different partitions cannot communicate
 - Each subgroup must be able to decide **independently** whether they are allowed to carry out operations
- A **quorum** is a subgroup of replicas whose size gives it the right to perform operations
 - For example, having the majority of replicas could be the criterion (in a system of 9 replicas, a subgroup with at least 5 replicas has the right to perform operations)

Gifford's Quorum-Based Protocol

- Update operations may be performed by **one subset of the replicas only**
 - → The other replicas may be out-of-date
 - Version numbers are used to determine which replicas are up-to-date
 - When a replica is updated, its version number is changed accordingly
 - Operations are applied only to replicas with the latest version number

Gifford's Quorum-Based Protocol

- Each read operation must obtain a read quorum $\geq R$ replicas before it can read from any up-to-date replica
- Each write operation must obtain a write quorum $\geq W$ replicas before it can proceed with an update operation
- R and W are set such that
 - $W > \text{half the total number of all replicas}$
 - $R + W > \text{the total number of all replicas}$
 - These constraints ensure that any pair of a read quorum and a write quorum or two write quorums contain common replicas

Gifford's Quorum-Based Protocol

- To perform a read operation, a client must assemble a read quorum
 - Contact a set of available replicas whose total number is not less than R
 - The set must contain at least one up-to-date replica
 - The read operation may be applied to any up-to-date replica in the read quorum (which can be figured out by comparing the version numbers of replicas)

Gifford's Quorum-Based Protocol

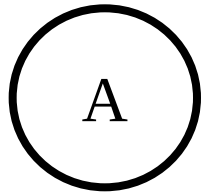
- To perform a write operation, a client must assemble a write quorum
 - Contact a set of available replicas whose total number is not less than W
 - The set must contain at least one up-to-date replica
 - If there are out-of-date replicas in the set, replace them with the up-to-date copy
 - The write operation is then applied to each replica in the write quorum, and the version number is incremented

Example of Gifford's Protocol

$x = 100$ (v0)



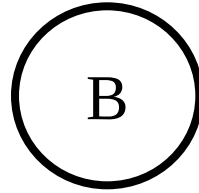
$x = 200$ (v1)



$x = 100$ (v0)



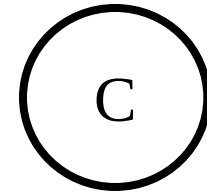
$x = 200$ (v1)



read(x) → 100

write($x = x + 100$)

$x = 100$ (v0)



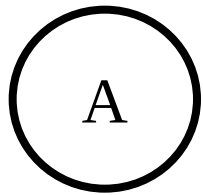
~~read(x)~~

~~write($x = x + 10$)~~

$R = W = 2$

Example of Gifford's Protocol

$x = 200$ (v1)



~~read(x)~~

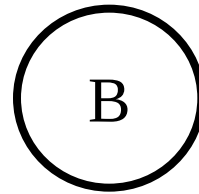
~~write(x=x+20)~~

$R = W = 2$

$x = 200$ (v1)



$x = 400$ (v2)



read(x) → 200

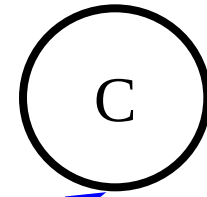
$x = 100$ (v0)



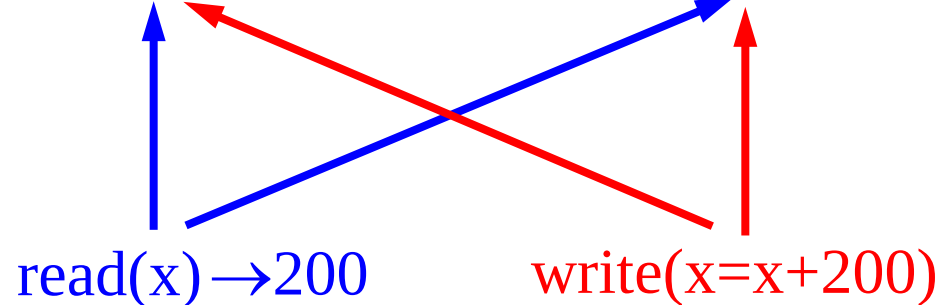
$x = 200$ (v1)



$x = 400$ (v2)



write(x=x+200)



Gifford's Quorum-Based Protocol

- There might be multiple eligible pairs of R and W
 - The system performance can be optimized by setting R and W appropriately under different situations
- Consider an object with three replicas
- If the object has a very high read to write ratio, we can set $R = 1$ and $W = 3$ to reduce traffic
 - Reads can be done at any replica, but writes must access all replicas (known as Read-One-Write-All)
- If the object has a very high write to read ratio, we can set $R = 2$ and $W = 2$ to reduce traffic
 - Writes need to access two replicas only (why not 1 replica?), and reads should also access two replicas

Outline

- Introduction
- Data-Centric Consistency Models and Protocols
- Client-Centric Consistency Models and Protocols

Client-Centric Consistency Models

■ Target scenario

- Lack of simultaneous updates (or when they happen, they can easily be resolved)
- Most operations involve reading data

■ Examples

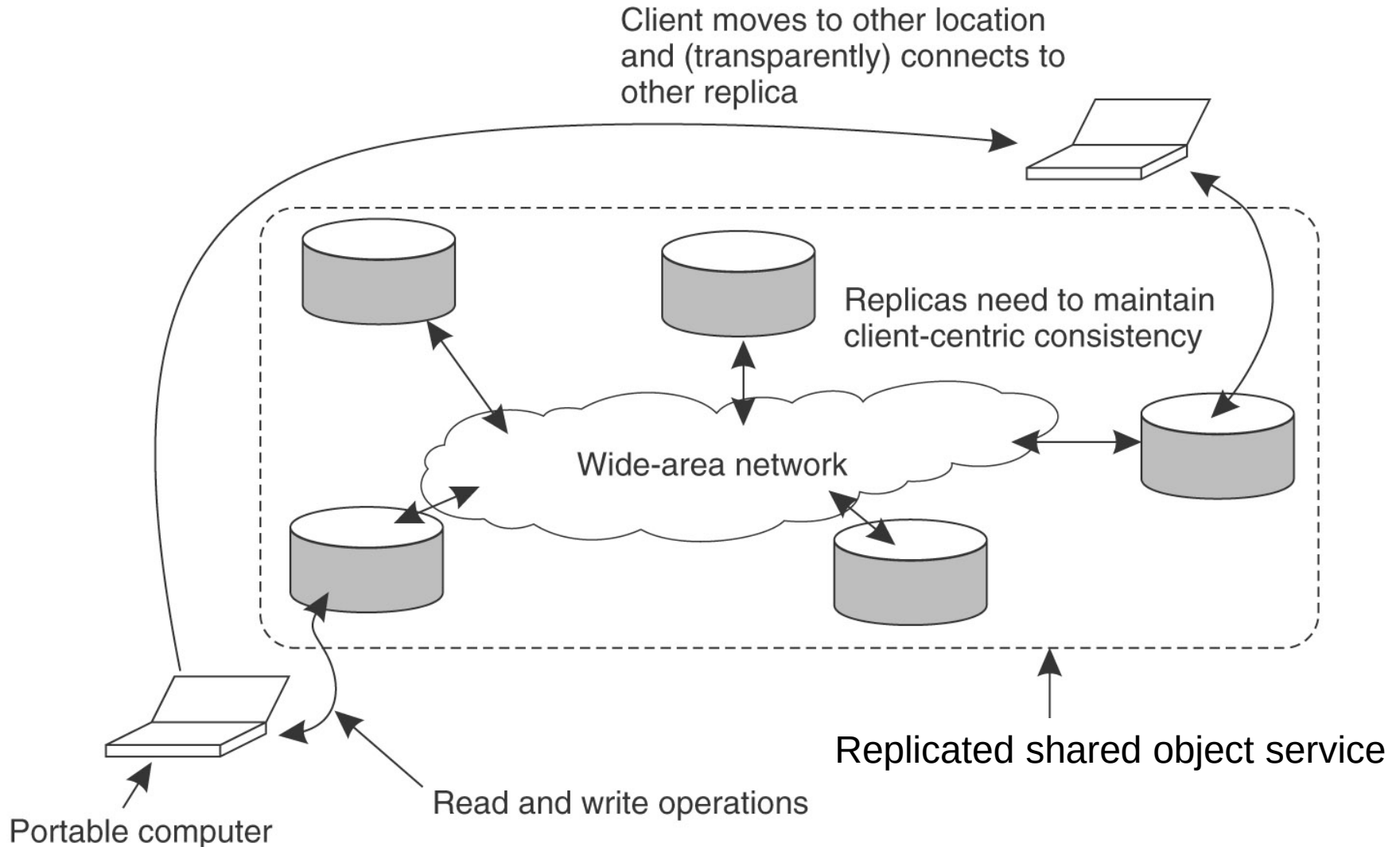
- In many database systems, only one or very few processes perform update operations
- In DNS, only the naming authority is allowed to update the name space
- In World Wide Web, web pages are normally updated by a web master only
- These applications can tolerate a relatively high degree of inconsistency

Client-Centric Consistency Models

- Eventual consistency

- If no update takes place for a long time, all replicas will gradually become consistent
- Essentially only require that updates are guaranteed to propagate to all replicas (possibly in a lazy fashion)
- Eventual consistency works fine as long as clients always access the same replica
- But problems arise when different replicas are accessed

Client-Centric Consistency Models



Client-Centric Consistency Models

- Client-centric consistency provides guarantees **for a single client** concerning the consistency of its accesses to data objects
- No guarantees are given concerning concurrent accesses by different clients

Client-Centric Consistency Models

- Monotonic reads
- Monotonic writes
- Read your writes
- Writes follow reads
- Examples in this section
 - In most cases, only one client updates an object
 - Updates are eventually propagated to all replicas
 - Initial value of an object x is o
 - Read operations $r_1, r_2, \dots$; write operations $w_1, w_2, \dots$
 - $r_2 \rightarrow o + w_1 + w_2$: a read operation on x returning a value integrating write operations w_1 and w_2

Monotonic Reads

- If a client reads the value of a data object x , any successive read operation on x by that client will always return that same value or a more recent value
- That is, if a client has seen a value of x at time t , it will never see an older version of x later
- Sample application: Distributed email database
 - Suppose a user reads his email in Singapore and then later flies to Hong Kong and opens his mailbox again
 - Messages that were in the mailbox in Singapore will also be in the mailbox when it is opened in Hong Kong

Monotonic Reads

- Example of a monotonic-read consistent service

Replica A	Replica B
	w_1 (become $o + w_1$)
propagate w_1 (become $o + w_1$)	
$r_1 \rightarrow o + w_1$	
	w_2 (become $o + w_1 + w_2$)
	$r_2 \rightarrow o + w_1 + w_2$
propagate w_2 (become $o + w_1 + w_2$)	

w_1 and w_2 may be performed by a different client from r_1 and r_2

Monotonic Reads

- Example of a not monotonic-read consistent service

Replica A	Replica B
	w_1 (become $o + w_1$)
propagate w_1 (become $o + w_1$)	
	w_2 (become $o + w_1 + w_2$)
	$r_1 \rightarrow o + w_1 + w_2$
$r_2 \rightarrow o + w_1$	
propagate w_2 (become $o + w_1 + w_2$)	

w_1 and w_2 may be performed by a different client from r_1 and r_2

Monotonic Writes

- A write operation by a client on a data object x is completed before any successive write operation on x by the same client
- In general, a write operation on a copy of x is performed only if that copy has been brought up to date by means of any preceding write operation, which may have taken place on other copies of x
 - Always necessary to bring the copy up to date first?
- Sample application: Software library
 - Updating the library is done by replacing functions
 - If an update is performed on a copy of the library, all preceding updates will be performed first

Monotonic Writes

- Example of a monotonic-write consistent service

Replica A	Replica B
w_1 (become $o + w_1$)	
	propagate w_1 (become $o + w_1$)
	w_2 (become $o + w_1 + w_2$)
	$r_1 \rightarrow o + w_1 + w_2$

r_1 may be performed by a different client from w_1 and w_2

Monotonic Writes

- Example of a not monotonic-write consistent service

Replica A	Replica B
w_1 (become $o + w_1$)	
	w_2 (become $o + w_2$)
	$r_1 \rightarrow o + w_2$

r_1 may be performed by a different client from w_1 and w_2

Read Your Writes

- The effect of a write operation by a client on a data object x will always be seen by a successive read operation on x by the same client
- That is, a write operation is always completed before a successive read operation by the same client, no matter where that read takes place
- Sample application: World Wide Web
 - A user's web browser caches a local copy of a web page after accessing it from a local web server
 - When the web page is updated at the server, the user will not see the new version if the browser returns the cached copy

Read Your Writes

- Example of a read-your-writes consistent service

Replica A	Replica B
	w_1 (become $o + w_1$)
propagate w_1 (become $o + w_1$)	
	w_2 (become $o + w_1 + w_2$)
propagate w_2 (become $o + w_1 + w_2$)	
$r_1 \rightarrow o + w_1 + w_2$	

w_1 , w_2 and r_1 are performed by the same client

Read Your Writes

- Example of a not read-your-writes consistent service

Replica A	Replica B
	w_1 (become $o + w_1$)
propagate w_1 (become $o + w_1$)	
	w_2 (become $o + w_1 + w_2$)
$r_1 \rightarrow o + w_1$	
propagate w_2 (become $o + w_1 + w_2$)	

w_1 , w_2 and r_1 are performed by the same client

Writes Follow Reads

- A write operation by a client on a data object x following a previous read operation on x by the same client is guaranteed to take place on the same or a more recent value of x that was read
- That is, any successive write operation by a client on a data object x will be performed on a copy of x that is up to date with the value most recently read by that client
- Sample application: Network newsgroup
 - Users of a network newsgroup see a posting of a reaction to an article only after they have seen the original article

Writes Follow Reads

- Example of a writes-follow-reads consistent service

Replica A	Replica B
w_1 (become $o + w_1$)	
	propagate w_1 (become $o + w_1$)
$r_1 \rightarrow o + w_1$	
	w_2 (become $o + w_1 + w_2$)

w_1 may be performed by a different client from r_1 and w_2

Writes Follow Reads

- Example of a not writes-follow-reads consistent service

Replica A	Replica B
w_1 (become $o + w_1$)	
$r_1 \rightarrow o + w_1$	
	w_2 (become $o + w_2$)

w_1 may be performed by a different client from r_1 and w_2

Client-Centric Consistency Protocols

- Each write operation is assigned a globally unique identifier
 - Can be assigned by the replica to which the write had been submitted (known as the origin of the write operation)
- Each client keeps track of two sets of write operations
 - Read set: (identifiers of) writes relevant to the read operations performed by the client
 - Write set: (identifiers of) writes performed by the client

Client-Centric Consistency Protocols

- To implement monotonic-read consistency
 - When a client performs a read operation at a replica, that replica is handed the client's read set to check whether all the identified writes have taken place locally
 - If not, the replica contacts other replicas to ensure that it is brought up to date for carrying out the read operation
 - Alternatively, the read operation is forwarded to a replica where the identified writes have already taken place
 - After the read operation is performed, the write operations that have taken place at the selected replica and which are relevant for the read operation are added to the client's read set

Client-Centric Consistency Protocols

- The write identifier could include the identifier of the replica to which the operation was submitted
- That replica could be required to log the write operation so that it can be replayed at another replica
- Other client-centric consistency models can be implemented in similar ways

Summary

- Replication helps services to provide good performance, reliability and fault tolerance
- Meanwhile, replication introduces the consistency problem
- Data-centric consistency models
 - Strict, sequential, causal and FIFO consistency
- Client-centric consistency models
 - Monotonic-read, monotonic-write, read-your-writes, and writes-follow-reads consistency